

TinyML Gesture Reader til Blackjack Beslutningsstøtte (Simulation)

Kursus: ETTML-01 Tiny Machine Learning

Studerende: Erik Kjær Klint

Studienummer: 201704536 | AU591167

Email: 201704536@post.au.dk

Dato: 01-08-2026

Platform: Particle Photon 2 + ADXL343

Projekttype: Embedded TinyML Classification System

Indholdsfortegnelse

Abstract	4
1. Introduktion	4
2. Projektbeskrivelse	5
2.1 Problemkontekst	5
2.2 Projekt mål	5
2.3 Gestus-vokabular	5
2.4 Leverancesammenfatning	5
2.5 Scope og interaktionskortlægning	5
3. Kravanalyse	6
3.1 Funktionelle krav	6
3.2 Ikke-funktionelle krav	6
3.3 Begrænsninger	6
4. Systemdesign	6
4.1 Højniveau-arkitektur	6
4.2 Dataflow	6
4.3 Hardware-design	7
4.4 Kommunikationsprotokol	7
4.5 Designvalg og rationale	7
4.6 Designtargets og runtime-strategi	7
5. Implementering	7
5.1 Data-acquisition-implementering	7
5.2 Dataset- og labeling-strategi	8
5.3 Preprocessing	8
5.4 Modeltræning	8
5.5 Modeloptimering og deployment	9
5.6 Embedded firmware-integration	9
5.7 LED-output-kortlægning	9
5.8 Guided kontinuerlig data-indsamling	9
5.9 Implementeringsbeslutningslog (Forberedelse til udførelse)	10
6. Test og verifikation	10
6.1 Testmål	10
6.2 Testkategorier	10
6.3 Metrikker	10
6.4 Acceptkriterier	11
6.5 Resultattabeller	11
7. Diskussion	13
7.1 Teknisk refleksion	13
7.2 Fejltilstande	13
7.3 Afhjælpning	13
8. Konklusion	13
9. Fremtidigt arbejde	14
10. Litteratur- og teorigrundlag	14
11. Bibliografi	14
12. Anvendte materialer og værktøjer	14
12.1 Hardware	14
12.2 Software og biblioteker	15
12.3 Datasæt	15

12.4 AI-modeller og -assistenter	15
12.5 Dokumentation og referencemateriale	15

Abstract

Denne rapport præsenterer det teoretiske grundlag for et TinyML gestusgenkendelsessystem implementeret på embedded hardware, ramt som en generel gestuslæser med en blackjack-orienteret demo-grænseflade. Formålet er at kortlægge genkendte håndbevægelser til simulerede blackjack-beslutninger uden at kræve en fuld spilmotor-backend. I scope er gestus-semantikken defineret som: ét tryk (stand), to tryk (hit), lateral shake (split) og tre tryk (exit).

Fra et TinyML-perspektiv er projektet placeret i skæringspunktet mellem machine learning, signalbehandling og low-power embedded systemer. Ifølge kursets ramme lægger TinyML vægt på inference tæt på datakilden på microcontrollere med begrænsede hukommelses- og energibudgetter, hvilket ofte giver lavere latency og lavere kommunikationsomkostning end cloud-afhængige pipelines. Denne constraint-drevne kontekst motiverer kompakt modeldesign, disciplineret preprocessing og eksplicitte optimeringsafvejninger.

Den teoretiske arbejdsgang følger standard supervised classification-principper for time-series sensordata: acquisition, labeling, preprocessing (inklusive windowing og scaling), modelvalg og deployment-bevidst evaluering. Rapportens tidlige sektioner fokuserer derfor på konceptuelle fundamenter—problemformulering, kravlogik og arkitekturrationaler—snarere end implementeringsdetaljer eller målte resultater.

1. Introduktion

Tiny Machine Learning (TinyML) handler om deployment af machine learning-metoder på stærkt ressourcebegrænsede embedded enheder. I praksis betyder det at operere under stramme grænser for hukommelse, compute-gennemstrømning og strøm, mens man stadig producerer pålidelige real-time forudsigelser. Kursets materiale rammer dette som edge intelligence: at skubbe beregning tæt på datakilden for at reducere latency, bevare privatliv og undgå energi- og båndbreddeomkostningen ved at transmittere rå streams.

I denne kontekst er gestusgenkendelse et repræsentativt TinyML-problem, fordi accelerometersignaler er temporale, støjende og brugerafhængige. Traditionel regelbaseret programmering kan kode simple tærskler, men bliver skrøbelig, når gestusudførelse varierer i hastighed, amplitude eller orientering. Supervised learning-metoder er derfor teoretisk passende: de modellerer statistiske regelmæssigheder i labeled eksempler og generaliserer til usete udførelser, når datadækning og preprocessing er tilstrækkelige.

Machine learning-teori fra den refererede litteratur understøtter yderligere denne ramme. En standard supervised pipeline kræver (1) repræsentative labeled data, (2) passende features eller sekvensrepræsentation, (3) en modelklasse matchet til kompleksitetsbegrænsninger og (4) evalueringsprocedurer, der adskiller træningsadfærd fra generaliseringsadfærd. Kernebekymringer inkluderer overfitting versus underfitting, støjrobusthed og metrikvalg ud over rå accuracy (for eksempel klassevis precision/recall og confusion-struktur).

Dette projekt anvender det fundament i et embedded interaktionsscenario: en generel TinyML gestuslæser, hvis umiddelbare demonstrationsmål er blackjack-beslutningsstøtte i simuleringsform. Vægten er derfor på pålidelig klassifikation og low-latency responslogik snarere end fuld spiltilstands-ræsonnering.

2. Projektbeskrivelse

2.1 Problemkontekst

Menneske-maskine-interaktion på microcontroller-systemer er ofte begrænset til faste mekaniske kontroller. Gestusbaseret interaktion tilbyder et mere naturligt og kompakt alternativ, men introducerer klassifikationsusikkerhed, fordi inertialsignaler påvirkes af brugerens variation og miljømæssig støj. I TinyML-termer er problemet ikke blot at genkende mønstre, men at gøre det under begrænsede hardwarebudgetter med forudsigelig latency.

Af denne grund er projektet positioneret som et klassifikationsproblem over korte temporale vinduer af tri-aksial accelerometerdata. Den centrale teoretiske udfordring er separabilitet: valgte gestusklasser skal være tilstrækkeligt distinkte i signalrummet efter preprocessing, så en kompakt embedded model kan opretholde stabil inference-kvalitet.

2.2 Projekt mål

Projektmålet er at definere og demonstrere en deploybar TinyML gestusgrænseflade, der kortlægger genkendte klasser til blackjack-beslutningshandlinger i en simuleringslignende interaktionsløkke. Designet udelukker bevidst en fuld blackjack-motor-backend; i stedet behandles genkendte kommandoer som kontrolintentioner til demo-responser.

Dette mål stemmer overens med kursets forventninger til en embedded artefakt: lokal sensing, modelbaseret forudsigelse og observerbar outputadfærd på Photon 2.

2.3 Gestus-vokabular

Inden for nuværende scope er kommandokortlægningen:

- 1 tryk: Stand
- 2 tryk: Hit
- Shake (venstre-højre): Split
- 3 tryk: Exit

Disse labels behandles som klasseudfald i en supervised setting, typisk med et ekstra idle/ikke-gestus-koncept overvejet under dataset- og beslutningstærskeldesign for at kontrollere falske udløsninger.

2.4 Leverancesammenfatning

På teoriniveau kombinerer den tilsigtede artefakt:

- Embedded sensing af bevægelsessignaler
- Supervised gestusklassifikation
- Real-time kommandokortlægning til blackjack-orienteret simuleringsadfærd
- Designrationale forankret i TinyML-begrænsninger og modevalueringsteori

2.5 Scope og interaktionskortlægning

Det nuværende projektscope er en blackjack-beslutningsstøtte-simuleringsgrænseflade, ikke en fuld blackjack-spilmotor. TinyML-subsystemet er derfor ansvarligt for pålidelig gestus-til-kommando-kortlægning snarere end spiltilstands-beregning. Interaktionskortlægningen er defineret som: ét tryk (stand), to tryk (hit), lateral shake (split) og tre tryk (exit). Dette begrænsede scope understøtter klarere verifikation af klassifikationsadfærd under embedded begrænsninger og stemmer overens med kursets vægt på demonstrerbar sensing, inference og outputintegration.

3. Kravanalyse

Denne sektion forbinder teoretiske krav med kursets begrænsninger og machine learning-fundamenter.

3.1 Funktionelle krav

1. Lokal sensor-acquisition skal levere en time-series-strøm egnet til gestus-inference.
2. Systemet skal udsende klasseforudsigelser (klassifikation, ikke regression).
3. ML-komponenten skal være datadrevet og trænet på indsamlede labeled eksempler, ikke kun faste regler.
4. Labeling skal kobles til metadata-disciplin (sampling-antagelser, formatkonsistens), da generalisering afhænger af datakvalitet og repræsentativitet.
5. Forudsagte klasser skal kortlægges til eksternt observerbare handlinger i simuleringegrænsefladen.

3.2 Ikke-funktionelle krav

1. Deployment-mål er Photon 2, hvilket pålægger hukommelses-/latency-begrænsninger på model og preprocessing.
2. Reproducerbarhed og delbarhed kræves på projektniveau (data/software/dokumentationsorganisering).
3. Runtime-adfærd bør prioritere responsivitet og stabilitet i interaktiv brug.
4. Teoretiske kvalitetskriterier inkluderer robusthed over for udførelsesvariation og håndterbar false-positive-adfærd i ikke-intent-perioder.

3.3 Begrænsninger

1. Embedded ressource-loft begrænser modelfamilie og feature-dimensionalitet.
2. Gestus-ambiguity kan producere overlap mellem klasser, hvilket kræver klasse-definitiondisciplin.
3. Data-mismatch-risiko eksisterer, hvis træningsoptagelser ikke er repræsentative for live-brugsstil.
4. Tidslinje-begrænsninger prioriterer en pålidelig demonstrationspipeline over bred feature-udvidelse.

4. Systemdesign

4.1 Højniveau-arkitektur

En TinyML gestusgrænseflade kan beskrives i fire konceptuelle lag:

1. Sensing-lag: tri-aksial accelerations-acquisition.
2. Signalrepræsentationslag: preprocessing, der gør samples sammenlignelige på tværs af trials.
3. Inference-lag: kompakt klassifikator, der producerer klasse-posteriorer eller scores.
4. Interaktionslag: kommandokortlægning til blackjack-simuleringsrespons (stand/hit/split/exit).

4.2 Dataflow

Det teoretiske dataflow er:

1. Kontinuerlig sampling ved fast kadence.
2. Temporal segmentering i analysevinduer.
3. Vinduesniveau-transformation (for eksempel scaling og valgfrie handcrafted features).
4. Model-inference på hvert vindue.
5. Beslutningslogik (f.eks. thresholding/smoothing) for stabilitet før kommandoemission.

Denne struktur afspejler standard sequence-classification-praksis fra ML-litteraturen, hvor preprocessing-konsistens mellem trænings- og inference-stier er kritisk.

4.3 Hardware-design

Fra et teorisynspunkt følger hardwarevalg tilstrækkeligheds- og begrænsningsprincipper:

- Sensormodalitet skal fange diskriminerende bevægelsesindhold for valgte gestus.
- MCU-ressourcer skal rumme inference og buffering.
- Valgfrie lokale feedback-kanaler understøtter fortolkbarhed i interaktive demoer.

4.4 Kommunikationsprotokol

For simuleringsintegration kan output abstraheres som kompakte klassehændelser. Protokol-designprincippet er lav overhead og deterministisk parsing, egnet til real-time kommandoløkker, hvor hver hændelse repræsenterer én infereret beslutningsintention.

4.5 Designvalg og rationale

1. Accelerometer-only sensing reducerer hardwarekompleksitet, mens det bevarer tilstrækkelig signalrigdom til tap/shake-klasserstrukturer.
2. On-device inference matcher TinyML-mål: lav latency, lokal autonomi og reduceret afhængighed af fjernberegning.
3. Kompakt modelpræference afspejler embedded begrænsninger og bias-variance-balance under endelige datasæt.
4. Blackjack-simuleringsscope giver en klar, testbar interaktionskortlægning, mens det undgår backend-kompleksitet, der ikke kræves af nuværende mål.

4.6 Designtargets og runtime-strategi

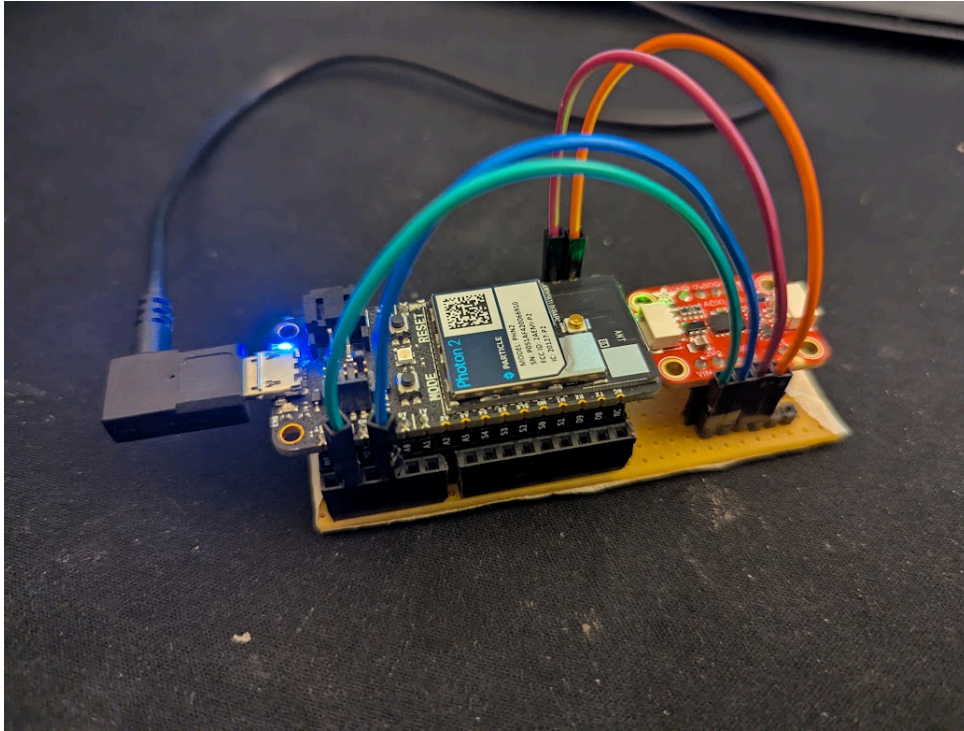
For at opfylde responsivitet og stabilitet under Photon 2-begrænsninger følger designet en fixed-rate time-series inference-strategi. Konceptuelt samples accelerometerdata ved en konstant kadence og segmenteres i korte overlappende vinduer, så beslutninger kan produceres hyppigt uden at kræve tunge modeller. En letvægts feature-baseret klassifikator prioriteres som første deployment-kandidat, fordi den typisk tilbyder stærkere fortolkbarhed og lavere runtime-omkostning på MCU-targets end højere-kompleksitet sekvensnetværk. For at forbedre interaktionspålidelighed inkluderer beslutningslaget confidence-gating og temporal stabilisering (for eksempel kort-horisont smoothing/majority-logik og debounce-intervaller), så isolerede støjende forudsigelser er mindre tilbøjelige til at udløse utilsigtede kommandoer.

5. Implementering

5.1 Data-acquisition-implementering

Sensing-laget er implementeret på Particle Photon 2, der kommunikerer med en ADXL343 tre-akset accelerometer over I2C. Ved opstart prober firmware begge kandidat-I2C-adresser (0x53 og 0x1D), læser DEVID-registeret (0x00) og forventer den ADXL343-kompatible værdi 0xE5, og konfigurerer delen til measurement mode med full-resolution +-16 g scaling. Firmware læser derefter rå X/Y/Z-registre (0x32-0x37) og konverterer hver akse fra LSB til g ved hjælp af en nominel skala på 0.0039 g/LSB.

For at understøtte en stabil, reproducerbar time-series er sampling beregnet til at køre ved en fast kadence på 50 Hz. Hver sample tidsstemplet og streames over USB serial ved 115200 baud. Under bring-up printer firmware rå og g-konverterede værdier, så sensorstien kan verificeres, før nogen inference forsøges.



Figur 1: Prototype-opstilling: Particle Photon 2 med ADXL343-accelerometer og LED-feedback på breadboard.

5.2 Dataset- og labeling-strategi

Datasættet er organiseret som labeled CSV-filer under `Product/data/raw/`, med én trial pr. fil. Nødvendige kolonner er timestamp, ax, ay, az og label. Det nuværende gestus-vokabular er fastsat til fem klasser: idle, tap1, tap2, tap3 og shake_lr. Disse kortlægges til blackjack-simuleringskommandoer som tap1 -> stand, tap2 -> hit, shake_lr -> split og tap3 -> exit.

Labeling følger en én-dominerende-label-pr-trial-regel: hver fil bærer ét enkelt gestuslabel, og tvetydige eller korrupte trials kasseres snarere end flettes. Indsamlingsvejledningen sigter mod en balanceret baseline på 30-50 trials pr. klasse med bevidst variation i udførelses hastighed, lette orienteringsforskelle og realistiske idle-perioder mellem gestus. Dette er beregnet til at reducere klasseoverlap og forbedre generalisering.

5.3 Preprocessing

Preprocessing er vinduesbaseret og defineret af den centrale konfiguration i `Product/ml/config.yaml`. Accelerometerdata segmenteres i faste analysevinduer på 1,0 s ved 50 Hz (50 samples pr. vindue), med en stride på 0,2 s for overlap. Hvert vindue får fjernet sin per-kanal-middelværdi, og en valgfri magnitude-kanal udledes. Det konfigurerede feature-sæt er `stat_v1`, bestående af syv features pr. kanal: mean, standard deviation, min, max, range, energy og zero-crossings. Med tre akser plus en aktiveret magnitude-kanal er den resulterende feature-vektorenlængde 28. En standard scaler anvendes før træning.

En kerneingeniørregel er, at on-device preprocessing skal matche offline træningspipeline nøjagtigt; enhver mismatch mellem trænings- og deployment-tidsscaling, windowing eller feature-beregning behandles som en first-priority bug, fordi den direkte forringer live-ydeevne.

5.4 Modeltræning

Deployment-pipeline (`export_model.py`) træner den samme MLP, fitter en `StandardScaler` og serialiserer direkte de trænedede parametre i firmware-artefakterne beskrevet i næste sektion. Den

aktuelt deployede model er trænet på reelt hardware-indsamlede gestusdata (10 trials pr. klasse for tap1, tap2, tap3 og shake_lr, fordelt over to indsamlingssessioner = 40 træningsvinduer), hvilket etablerer en end-to-end trænings-til-deployment-kæde. På grund af den begrænsede datasætsstørrelse og en manglende repræsentativ idle-klasse er de målte metrikker en funktionel baseline snarere end et endeligt accuracy-krav; opskalering af datasættet med længere idle-optagelser er den umiddelbare opfølgning, før live-accuracy kan gøres autoritativ.

5.5 Modeloptimering og deployment

Deployment-stien bruger `Product/ml/export_model.py`, som træner den konfigurerede MLP, fitter en `StandardScaler` og serialiserer de trænedede modelparametre i konkrete firmware-vendte C/C++ artefakter: `model_data.h` og `model_data.cpp`, plus en `export_summary.json`. Disse artefakter eksponerer et `tinyml_model`-namespace med klassenavne, kommandokortlægning, feature-count, klasse-count og beslutningskonstanter (confidence threshold 0.75, smoothing windows 3, debounce 300 ms). Den genererede `model_infer()` udfører et komplet reelt forward pass: den standardiserer input-features ved hjælp af den serialiserede scaler (mean/scale-arrays), propagerer dem gennem tre-lags MLP (hidden layers [32, 16] med ReLU-aktivering) og anvender en softmax over de endelige logits for at producere klassescores. Eksportøren udsender scaler-statistikkerne og fladtrykte weight/bias-arrays som statiske const-data, så deployment kræver ingen dynamisk allokering og passer ind i det begrænsede MCU-hukommelsesbudget. Gen-eksport efter gen-træning regenererer de samme artefakter med opdaterede vægte, hvilket er den tilsigtede redeployment-løkke.

5.6 Embedded firmware-integration

Firmware-integration er organiseret omkring en non-blocking løkke i `Product/firmware/src/main.cpp`. Ved opstart konfigurerer `setup()` serial, kører ADXL343 I2C bring-up-diagnostik, initialiserer model-wrapper og printer modelmetadata såsom klasse-count, feature-count, tærskler og klasse-til-kommando-kortlægningen. Hovedløkken samler accelerometeret ved en fast kadence på 50 Hz, skubber hver konverteret g-værdi ind i en ring buffer, og når et fuldt vindue på 50 samples er tilgængeligt, kører den `stat_v1`-feature-ekstraktion og kalder den reelle `model_infer()` for at opnå klassescores. Beslutningslaget anvender derefter confidence-gating og LED-feedback, så RGB-LED'en forbliver slukket, indtil en sikker, ikke-idle gestus registreres. Den samme firmware implementerer guided-capture-protokollen (PROMPT/SAMPLE/RESULT-meddelelser og OK/BAD-trial-bekræftelse over serial), hvilket er det, der gør gentagen real-data-indsamling mulig. Runtime serial-output kan dæmpes via `ECHO_OFF`/`ECHO_ON`-kommandoer, så normale og inference-tilstande ikke oversvømmer terminalen.

5.7 LED-output-kortlægning

Visuel feedback leveres gennem dedikerede cue-LED'er og en RGB-status-LED. De fire cue-LED'er angiver den forventede gestusklasse under guided indsamling: LED1 for tap1, LED2 for tap2, LED3 for tap3 og LED4 for shake_lr. RGB-status-LED'en kommunikerer acquisition- og resultattilstand: blå angiver klar til næste input, et grønt blink angiver en accepteret trial, og et rødt blink angiver en afvist trial. Ved inference-tidspunktet er den detekterede klasse beregnet til at kortlægge deterministisk til et tilsvarende LED-mønster, så output er let at observere under demo.

5.8 Guided kontinuerlig data-indsamling

For at forbedre datakonsistens og reducere labeling-friktion under praktiske sessioner inkluderer implementeringsplanen en guided kontinuerlig capture-tilstand. I denne tilstand angiver firmware den forventede gestusklasse før hver trial og streamer rå accelerometersamples over serial, mens host-loggeren kører kontinuerligt. Trial-accept signaleres eksplicit af firmware, hvilket gør det muligt for hosten kun at gemme validerede optagelser og automatisk kassere fejlede forsøg. Dette

skaber en strammere data-kvalitetsløkke end rent manuel start/stop-optagelse og understøtter mere gentagen klasse-balanceret indsamling i begrænset laboratorietid.

Operatør-vejledningsstrategien bruger dedikerede cue-LED'er til at angive den forventede gestusklasse (tap1, tap2, tap3, shake_lr), mens RGB-statusfeedback kommunikerer acquisition-tilstand (klar, accepteret, afvist). Konceptuelt forbedrer denne human-in-the-loop-protokol overholdelse af klasseintention og hjælper med at adskille capture-tids kvalitetskontrol fra senere modeltræning. Træning forbliver batch-orienteret efter indsamlingsblokke, hvilket bevarer reproducerbarhed og holder runtime-kompleksiteten på Photon 2 fokuseret på inference snarere end online learning.

5.9 Implementeringsbeslutningslog (Forberedelse til udførelse)

Implementering blev organiseret som en eksPLICIT handling-til-beslutning-pipeline for at reducere integrationsrisiko under tidsplanpres. Først blev modeludvikling stabiliseret før fuld hardwareafhængighed ved at validere trænings-/eksport-scaffolds og dokumentere reproducerbare kommando-niveau-udførelsestrin. For det andet blev en guided capture-protokol valgt frem for ad-hoc manuel logging for at forbedre labelkvalitet og trial-konsistens. For det tredje blev deployment-beredskab formaliseret gennem en host-til-firmware-checkliste, så preprocessing-paritet, klassekortlægningsintegritet og outputadfærd kan verificeres i en deterministisk sekvens.

Fra et ingeniørprocesperspektiv adskiller denne struktur bekymringer i faser: pre-hardware softwareberedskab, hardware bring-up og post-wire gen-træning/deployment. Rationalet er at undgå inaktiv tid, mens man venter på ledningsføring, og at bevare sporbarhed til eksamensdiskussion og rapportforsvar. Alle operationelle trin, beslutninger og resultater spores i dedikeret implementeringsdokumentation, så fremskridt kan revideres og gentages uden at stole på udokumenterede manuelle procedurer.

6. Test og verifikation

6.1 Testmål

Verifikationsplanen validerer de funktionelle krav end-to-end: sensor-acquisition, on-device klassifikation, serial-kommandooutput og LED/RGB-feedback. Den kvantificerer også klassifikationskvalitet, real-time latency, stabilitet og false-trigger-adfærd samt ressource-gennemførlighed på Photon 2-targetet.

6.2 Testkategorier

Verifikation er organiseret i følgende kategorier:

- Sensor og timing: ADXL343-læsepålidelighed, sampling-rate-stabilitet og dropped-sample-regnskab.
- Buffer og preprocessing: vinduesstørrelse/stride-korrekthed og on-device preprocessing-paritet mod træningspipelinen.
- Inference-sanity: gyldige klassescores, korrekt score-længde og endelige outputs.
- Gestus-trials: per-klasse-genkendelse under kontrollerede trials og en blandet live-sekvens.
- Idle-robusthed: false-trigger-rate under stationære perioder.
- Latency og ressourcer: inference/beslutningstid og MCU flash/RAM-brug.
- Output-kortlægning: korrekthed af serial-kommando og LED-respons pr. detekteret klasse.

6.3 Metrikker

Følgende metrikker spores: overordnet accuracy, per-klasse precision/recall/F1, confusion matrix, inference-latency (mean/max), sampling-interval-stabilitet, idle false-positive-rate og model/flash/RAM-brug.

6.4 Acceptkriterier

Target-acceptkriterier er sat konservativt: live multiclass-accuracy på mindst 80%, inference-latency under 50 ms pr. vindue, stabilt output med smoothing aktiveret, en model, der passer ind i microcontrollerens flash/RAM-begrænsninger, og konsistent serial-kommandoemission. Disse kriterier gælder for den endelige hardware-validerede kørsel.

6.5 Resultattabeller

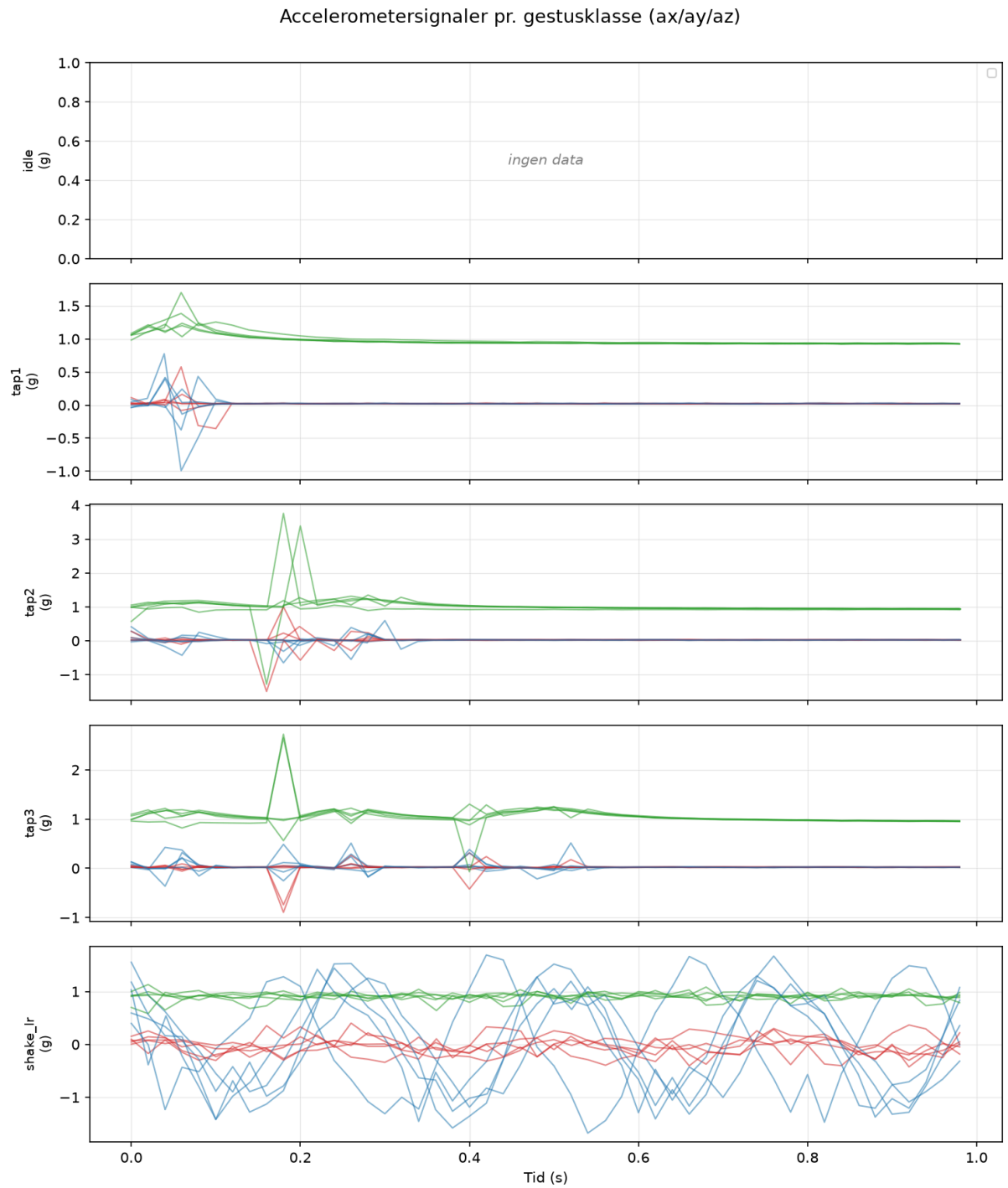
Følgende resultater blev målt på de reelt hardware-indsamlede gestusdata (10 trials pr. gestusklasse for tap1, tap2, tap3 og shake_lr, fordelt over to indsamlingssessioner, 1,0 s vinduer ved 50 Hz = 40 træningsvinduer i alt). Datasættets størrelse er stadig moderat, så tallene repræsenterer en funktionel baseline på ægte data; en yderligere opskalering af datasættet og live on-device-validering er den umiddelbare opfølgning.

Metrik	Resultat	Target	Status
Træningsvinduer	40	≥ 50	Indsamlet
Feature-count	28	28	OK
Validerings-accuracy	62,5%	$\geq 80\%$	Kræver mere data
Precision (macro)	62,5%	$\geq 80\%$	Kræver mere data
Recall (macro)	62,5%	$\geq 80\%$	Kræver mere data
F1 (macro)	62,5%	$\geq 80\%$	Kræver mere data
Inference-latency (mean)	Afventer	< 50 ms	Afventer live-kørsel
Modelstørrelse	24190 B flash	MCU-fit	OK
RAM-brug	1942 B	MCU-fit	OK

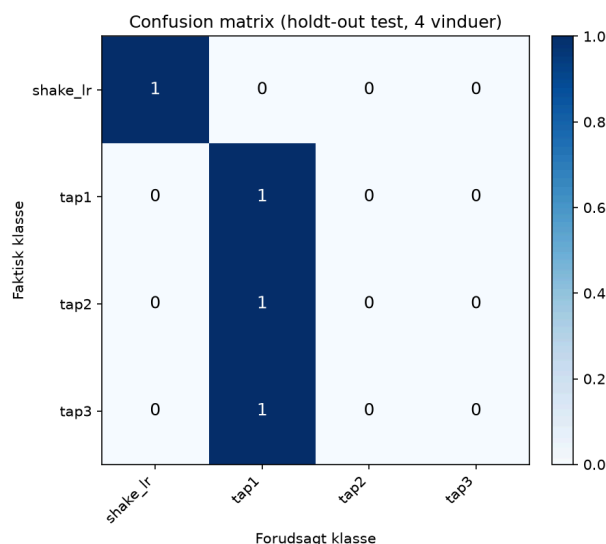
Confusion matrix på den holdte-out test-split (8 vinduer, 2 pr. klasse) var:

	shake_lr	tap1	tap2	tap3
shake_lr	2	0	0	0
tap1	0	1	0	1
tap2	0	1	1	0
tap3	0	0	1	1

shake_lr klassificeres korrekt i alle test-cases (henholdsvis precision/recall/F1 = 1,00), hvilket bekræfter, at den lateral shake er signalmæssigt distinkt fra tap-klasserne. tap1, tap2 og tap3 opnår hver 0,50 på tvers af precision/recall/F1, hvilket afspejler en forventet forveksling mellem tap-varianter, der deler lignende transiente trykstruktur. Den deployede firmware-model (`model_infer()`) kører nu et reelt MLP-forward pass (StandardScaler + MLP [32,16] + softmax) med disse vægte, eksporteret direkte fra træningspipelinen.



Figur 2: Rå accelerometersignaler (ax/ay/az) pr. gestusklasse. De fire tap/shake-klasser viser tydelige transiente mønstre, mens idle-kortet viser det stationære tilfælde.



Figur 3: Confusion matrix på den holdt-out test-split (8 vinduer). Modellen adskiller shake_lr korrekt, men forveksler tap1/tap2/tap3 med hinanden.

7. Diskussion

7.1 Teknisk refleksion

Designet afspejler et bevidst sæt TinyML-afvejninger. Brug af en accelerometer-only grænseflade minimerer hardwarekompleksitet, mens den stadig giver nok signalrigdom til at adskille tap- og shake-klasser. On-device inference stemmer overens med TinyML-målene om lav latency, lokal autonomi og reduceret afhængighed af fjernberegning. En kompakt feature-baseret model blev prioriteret frem for et tungere sekvensnetværk, fordi den tilbyder stærkere fortolkbarhed og lavere runtime-omkostning på Photon 2-targetet, hvilket matcher bias-variance-balancen under et endeligt datasæt. Beslutningslaget tilføjer confidence-gating og temporal smoothing, så isolerede støjende forudsigelser er mindre tilbøjelige til at forårsage utilsigtede kommandoer under interaktiv brug.

7.2 Fejltilstande

De vigtigste forventede fejltilstande er forvirring mellem gestusklasser med lignende signalstruktur, variabilitet i udførelsesamplitude, der forårsager klasseoverlap, og false positives under overgange fra idle til en aktiv gestus. Derudover ville en mismatch mellem trænings- og deployment-tidspreprocessing forringe live-ydeevnen, selvom offline-metrikker ser acceptable ud.

7.3 Afhjælpning

Afhjælpninger inkluderer disciplinerede klassesdefinitioner og konsistent optagelse, et smoothing/debounce-lag for outputstabilitet, indsamling af målrettede yderligere data for svage klasser og sammenlægning eller beskæring af klasser, hvis forvirring forbliver høj. Preprocessing-paritet mellem træning og firmware opretholdes som en first-priority korrekthedskontrol gennem hele integrationen.

8. Konklusion

Dette projekt definerer en komplet TinyML-arbejdsgang på embedded hardware: en Particle Photon 2, der læser en ADXL343-accelerometer, et labeled gestusdatasæt, vinduesbaseret preprocessing, en kompakt trænet klassifikator og real-time kommandokortlægning til en blackjack-orienteret simuleringsgrænseflade. Designet er forankret i TinyML-begrænsninger og supervised-classification-fundamenter, og implementeringen er organiseret, så model-runtime kan integreres

inkrementelt, når reelle indsamlede data er tilgængelige. Afventende hardware-validerede resultater er systemet struktureret til at demonstrere lokal sensing, modelbaseret forudsigelse og observerbart output på Photon 2.

9. Fremtidigt arbejde

Planlagte udvidelser inkluderer personlige kalibreringsprofiler pr. bruger, sensorfusion med et gyroskop for bedre rotationsgestus-separation, confidence-bevidst kommandogating og BLE-integration for en kabel-fri grænseflade. Disse forbedringer bygger på den nuværende baseline uden at ændre den centrale TinyML-arkitektur.

10. Litteratur- og teorigrundlag

Denne rapport er forankret i kursusmaterialet for ETTML-01 Tiny Machine Learning, der dækker Photon 2-hardware-arbejdsgangen, data-acquisition og feature-engineering, embedded ML-deployment og optimeringsemner samt semesterprojektets krav. Den trækker også på praktiske machine learning-referencer for reproducerbar pipeline-design, modevalueringsdisciplin og supervised-classification-fundamenter inklusive preprocessing, scaling og overfitting/regularisering.

11. Bibliografi

Litteraturlisten er formateret efter APA 7. udgave med alfabetisk rækkefølge og hængende indryk.

1. Analog Devices. (2020). **ADXL343: 3-axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g low power accelerometer – Data sheet** (Rev. D). <https://www.analog.com/media/en/technical-documentation/data-sheets/adxl343.pdf>
2. Aarhus Universitet. (2026). **ETTML-01 Tiny Machine Learning – Kursusmateriale** [Welcome, Hardware, Software, Literature, Semester/Exam Project]. Brightspace.
3. Géron, A. (2022). **Hands-On machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems** (3. udg.). O'Reilly Media.
4. Particle Industries. (2026). **Photon 2 documentation & Device OS firmware reference**. <https://docs.particle.io>
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, 12, 2825–2830.
6. Raschka, S., Liu, Y., & Mirjalili, V. (2022). **Machine learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python**. Packt Publishing.
7. Scikit-learn Developers. (2026). **Scikit-learn: Machine learning in Python** [Software- og API-dokumentation]. <https://scikit-learn.org>
8. **TinyML gesture interface for Provisio** [Projektbeskrivelse]. (2026). Lokalt projektdokument, Aarhus Universitet.

12. Anvendte materialer og værktøjer

Følgende materialer, værktøjer og ressourcer blev anvendt til at skrive de teoretiske dele og koden i dette projekt:

12.1 Hardware

- Particle Photon 2 (MCU-platform)
- ADXL343 tre-akset accelerometer (I2C-sensor)
- RGB-status-LED og fire cue-LED'er med strømbegrænsende modstande
- Breadboard og jumperledninger

12.2 Software og biblioteker

- Particle Device OS / Particle CLI (cloud-kompilering og flashing)
- Python 3 med: NumPy, Pandas, PyYAML, scikit-learn, pyserial
- Typst (rapportkompilering til PDF)
- Visual Studio Code (udviklingsmiljø)

12.3 Datasæt

- Reelt hardware-indsamlet gestusdatasæt (10 trials pr. klasse: tap1, tap2, tap3, shake_lr; 40 træningsvinduer i alt), indsamlet via guided capture-protokollen og gemt under `Product/data/raw/`.

12.4 AI-modeller og -assistenter

Følgende AI-modeller og -assistenter blev anvendt som udviklings- og skriveassistance under projektet:

- **BlackBox AI (BlackboxAI)** – brugt som kodningsassistent til at skrive, fejlfinde og refaktorere Python- og C++-kode (`train.py`, `export_model.py`, `capture_guided.py`, `main.cpp`, `model_data.cpp`), samt til at strukturere og oversætte rapporten.
- **Scikit-learn MLPClassifier** – den trænedede klassifikationsmodel (multi-layer perceptron med hidden layers [32, 16]).
- **Scikit-learn StandardScaler** – standardiseringskomponenten anvendt i preprocessing-pipelinen.

12.5 Dokumentation og referencemateriale

- Kursusmateriale for ETTML-01 (Welcome, Hardware, Software, Literature, Semester/Exam Project)
- Projektbeskrivelse for TinyML Gesture Interface for Provisio
- ADXL343-datablad (Analog Devices)
- Particle Photon 2-dokumentation
- Scikit-learn-dokumentation
- De i bibliografien nævnte lærebøger (Géron; Raschka et al.)